Hands-On Lab 2 : Understanding Git Diff

Learning Objective

- Understand what `git diff` does.
- Learn how to compare changes in different states.
- Understand Working Directory vs Staging Area vs Commit.
- Learn practical diff use cases.

Learning Outcome

After completing this lab, you will be able to:

- Compare unstaged changes.
- Compare staged changes.
- Compare between commits.
- Use `git diff HEAD` effectively.

Concept Explanation

What is git diff?

`git diff` shows line-by-line differences between versions of files.

It helps answer: What exactly changed? What did I modify? What is different between two commits?

Types of git diff

1️⃣ Unstaged Changes

```
git diff app.py
```

Compares: Working Directory vs Staging Area

2️⃣ Staged Changes

```
git diff --staged
```

Compares: Staging Area vs Last Commit (HEAD)

3️⃣ Compare With HEAD

```
git diff HEAD app.py
```

Compares: Working Directory vs Last Commit

4️⃣ Compare Two Commits

```
git diff commit1 commit2
```

Shows difference between two commit snapshots.

Lab Steps

Step 1 : Create and Commit

```
vi app.py
git add app.py
git commit -m "first commit"
```

Step 2 : Modify File

```
vi app.py
```

Add new content.

Check difference:

```
git diff app.py
```

Observe:

- Lines with `+` are additions.
- Lines with `-` are deletions.

Step 3 : Stage Changes

```
git add app.py
```

Check staged difference:

```
git diff --staged
```

Step 4 : Make Another Change

```
vi app.py
```

Check difference from last commit:

```
git diff HEAD app.py
```

Step 5 : Compare Commits

```
git log --oneline
git diff commitID1 commitID2
```

Real World Use Cases

✔️ Code review ✔️ Debugging unexpected behavior ✔️ Before committing to ensure correct changes ✔️ Comparing branches

---

Key Understanding Table

Command	Comparison
git diff	Working vs Staging
git diff --staged	Staging vs HEAD
git diff HEAD	Working vs HEAD
git diff commit 1 commit 2	Commit vs Commit
